

Projets de Recherche Appliquée IA Agentique, Mémoire, Tools, RAG, MCP et A2A

Dossier officiel de sujets - version encadrée pour les étudiants

Auteur du module : Prof. Hajji Tarik

ENSAM Meknès - 4A - Représentation et Résolution de Problèmes en IA

Finalité du dossier : proposer 7 sujets de projets avancés, chacun centré sur un axe scientifique principal (architecture agentique, mémoire, tools, RAG, MCP, A2A, intégration globale), tout en imposant l'utilisation de la pile complète. Chaque équipe devra produire une problématique claire, une contribution argumentée, un prototype fonctionnel, un article de qualité, un code source commenté étape par étape et une présentation professionnelle.

Année universitaire 2025-2026

1. Cadre général des projets

Ces sujets ne doivent pas être traités comme de simples travaux pratiques d'ingénierie. Chaque projet doit être abordé comme une mini-démarche de recherche et développement, fondée sur une problématique réelle, une hypothèse de travail, une architecture proposée, une expérimentation comparative et une discussion critique des résultats.

Chaque équipe doit obligatoirement mobiliser la pile complète suivante : un LLM local (par exemple via Ollama), un mécanisme de tools, une mémoire de session et une mémoire persistante, un pipeline RAG, au moins un serveur MCP et une communication ou coopération inter-agents inspirée de A2A. En revanche, chaque projet se spécialise dans un axe particulier qui constitue sa dominante scientifique.

Le but n'est donc pas seulement de faire fonctionner un démonstrateur, mais de résoudre un problème technique ou méthodologique précis : améliorer la robustesse, réduire les hallucinations, optimiser le choix des tools, concevoir une mémoire plus pertinente, comparer plusieurs variantes de RAG, structurer des services MCP modulaires, ou encore étudier la coopération inter-agents.

2. Contraintes communes imposées à toutes les équipes

- Le projet doit présenter une problématique explicite, formulée sous forme de question de recherche ou de défi d'architecture.
- La solution doit intégrer la pile complète : LLM local, tools, mémoire, RAG, MCP et A2A ou simulation rigoureuse d'une logique A2A.
- Le code source doit être structuré, commenté et expliqué par étapes. Le dépôt doit comporter un README technique permettant la reproduction complète.
- Un article scientifique de qualité doit être produit. Il doit inclure : introduction, état de l'art synthétique, problématique, méthodologie, architecture, expérimentation, résultats, discussion, limites et perspectives.
- Une présentation orale professionnelle est obligatoire. Elle doit exposer la motivation, la démarche, l'architecture, les résultats expérimentaux et la valeur ajoutée de l'équipe.
- Une expérimentation comparative est obligatoire : comparaison d'au moins deux variantes, politiques ou configurations, avec métriques justifiées.
- Le corpus, les jeux de tests, les choix de modèles, les limites matérielles et les biais doivent être explicités.
- La contribution ne doit pas se limiter à l'intégration de bibliothèques ; un effort réel de conception, d'analyse et d'innovation est attendu.

3. Livrables attendus

Livrable	Contenu minimal	Exigence qualité	Pondération indicative
Article scientifique	8 à 15 pages, style académique, schémas et références	Qualité rédactionnelle, argumentation, méthodologie, résultats	30 %
Code source	Architecture claire, commentaires, scripts d'installation et de test	Reproductibilité, lisibilité, modularité, traçabilité	30 %
Présentation orale	12 à 18 slides, démonstration et discussion critique	Clarté, profondeur technique, professionnalisme	20 %

Prototype/démo	Scénario de test complet, mesures, journalisation	Fonctionnement réel et preuve expérimentale	20 %
----------------	---	---	------

Projet 1 - Orchestrateur multi-agents adaptatif pour l'assistance académique

Dominante scientifique : architecture agentique.

Problématique

Comment concevoir une architecture agentique capable de répartir intelligemment une tâche complexe entre plusieurs sous-agents spécialisés tout en conservant cohérence, traçabilité et qualité finale ? Les étudiants devront résoudre la difficulté classique des agents monolithiques : surcharge contextuelle, mauvaise spécialisation, faiblesse de planification et difficulté à vérifier les sorties.

Contexte applicatif conseillé

Le système cible peut être un assistant universitaire avancé capable de traiter des questions de cours, synthétiser des documents, proposer des plans de résolution, appeler des tools, rechercher des passages pertinents et produire une réponse finale argumentée.

Travail de recherche demandé

Étudier et comparer deux architectures d'orchestration : par exemple hiérarchique (chef d'orchestre + sous-agents) contre architecture en graphe ou peer-to-peer. Il faut justifier le choix des rôles, des flux de messages, du mécanisme de routage, du protocole de validation et de la stratégie de reprise sur erreur.

Dimension innovation attendue

La contribution attendue peut porter sur un routeur dynamique fondé sur le type de requête, le niveau d'incertitude, la charge contextuelle, le coût estimé ou le score de confiance. Une visualisation des décisions de routage est fortement recommandée.

Contraintes spécifiques

- Au moins 4 agents spécialisés : planification, retrieval/RAG, tools, vérification ou synthèse.
- Utilisation obligatoire d'une mémoire de session et d'une mémoire persistante.
- Au moins 1 serveur MCP et 1 interaction inter-agents inspirée de A2A.
- Mesurer latence, taux d'échec, qualité finale et stabilité selon le type de tâche.

Livrables spécifiques

Article comparatif sur les architectures, code orchestrateur complet, schémas d'architecture, étude expérimentale sur plusieurs catégories de questions.

Questions de recherche possibles

- Une architecture hiérarchique produit-elle des réponses plus robustes qu'une architecture distribuée ?
- Quels signaux peuvent piloter le routage intelligent entre agents ?

Critères d'évaluation prioritaires

Critère	Ce qui est attendu	Poids conseillé
Profondeur scientifique	Problématique claire, hypothèse, analyse critique	35 %
Qualité technique	Architecture, code, reproductibilité, intégration complète	35 %

Évaluation expérimentale	Comparaisons, métriques, interprétation	20 %
Communication	Article, figures, présentation, démonstration	10 %

Projet 2 - Mémoire hybride longue durée pour agents IA locaux

Dominante scientifique : mémoire.

Problématique

Comment doter un agent local d’une mémoire utile, sélective et durable sans dégrader la précision ni saturer le contexte ? Le projet doit traiter la gestion de souvenirs à court terme et à long terme, la consolidation, l’oubli et la récupération ciblée.

Contexte applicatif conseillé

Le système peut être un assistant pédagogique personnalisé qui retient les préférences, le niveau, les erreurs récurrentes, les projets précédents et l’historique de progression d’un étudiant.

Travail de recherche demandé

L’équipe doit concevoir une mémoire multi-couches : mémoire conversationnelle, mémoire épisodique, mémoire sémantique, mémoire de préférences, mémoire procédurale. Elle doit ensuite comparer plusieurs politiques d’écriture et de récupération.

Dimension innovation attendue

Approches possibles : résumé périodique, consolidation par score d’utilité, oubli contrôlé, mémoire par événements, retrieval mémoire guidé par intention, séparation entre mémoire utilisateur et mémoire tâche.

Contraintes spécifiques

- Stockage persistant réel : JSON, SQLite, base vectorielle locale ou combinaison justifiée.
- Présence obligatoire d’outils de save/load/update/delete memory.
- Comparaison d’au moins deux politiques de consolidation et d’oubli.
- Lien obligatoire entre mémoire et pipeline RAG.

Livrables spécifiques

Article orienté expérimentation mémoire, corpus de tests conversationnels, démonstrateur avec profils utilisateurs, analyses d’erreurs.

Questions de recherche possibles

- Quel type de mémoire améliore réellement la continuité conversationnelle ?
- Comment éviter la pollution mémoire et les souvenirs inutiles ?

Critères d’évaluation prioritaires

Critère	Ce qui est attendu	Poids conseillé
Profondeur scientifique	Problématique claire, hypothèse, analyse critique	35 %
Qualité technique	Architecture, code, reproductibilité, intégration complète	35 %
Évaluation expérimentale	Comparaisons, métriques, interprétation	20 %
Communication	Article, figures, présentation, démonstration	10 %

Projet 3 - Taxonomie et sélection dynamique de tools pour agents IA

Dominante scientifique : tools.

Problématique

Comment un agent peut-il choisir le bon outil au bon moment parmi une bibliothèque hétérogène de capacités locales et distantes ? Le projet doit dépasser la simple déclaration de fonctions et traiter la sélection, la priorisation, la sécurité et la robustesse.

Contexte applicatif conseillé

Le système cible est un agent outillé disposant de tools de calcul, accès fichiers, résumé, exécution Python, tool Java, recherche documentaire, validation de réponses, export documentaire et éventuellement outils spécialisés du domaine.

Travail de recherche demandé

Construire une typologie des tools et proposer une politique de sélection : par coût, par type de donnée, par fiabilité, par niveau de risque, par historique de succès ou par contexte courant.

Dimension innovation attendue

Une contribution intéressante consiste à maintenir une mémoire des performances de chaque tool, puis à orienter les appels selon cette expérience accumulée. Une couche de validation ou de fallback entre tools similaires peut être introduite.

Contraintes spécifiques

- Au moins 6 tools réels, dont un tool Java et un tool Python.
- Exposition des tools via MCP avec schémas clairs.
- Gestion des erreurs, validation des entrées et journalisation obligatoires.
- Étude comparative entre appel direct par prompt et sélection enrichie par méta-données.

Livrables spécifiques

Catalogue de tools, benchmark de sélection, architecture MCP des tools, rapport sur sécurité et fiabilité.

Questions de recherche possibles

- La sélection guidée par historique et méta-données surpasse-t-elle le tool calling naïf ?
- Quels critères doivent déterminer le choix d'un tool en environnement local ?

Critères d'évaluation prioritaires

Critère	Ce qui est attendu	Poids conseillé
Profondeur scientifique	Problématique claire, hypothèse, analyse critique	35 %
Qualité technique	Architecture, code, reproductibilité, intégration complète	35 %
Évaluation expérimentale	Comparaisons, métriques, interprétation	20 %
Communication	Article, figures, présentation, démonstration	10 %

Projet 4 - RAG hybride, contextualisé et comparatif sur corpus spécialisés

Dominante scientifique : RAG.

Problématique

Comment réduire les hallucinations et améliorer la pertinence des réponses dans un agent IA lorsqu'il s'appuie sur un corpus spécialisé, francophone et hétérogène ? Le projet doit traiter les limites des approches de récupération naïves.

Contexte applicatif conseillé

Le corpus peut regrouper cours, TD, TP, FAQ, documentations, normes, mémoires de projets ou rapports internes. Le système doit produire des réponses sourcées et explicables.

Travail de recherche demandé

Comparer plusieurs variantes : RAG lexical, dense, hybride, query rewriting, reranking, contextual retrieval, multi-hop, graph-RAG simplifié ou retrieval avec mémoire utilisateur.

Dimension innovation attendue

L'équipe doit proposer une stratégie de chunking, d'indexation et de sélection contextuelle adaptée au domaine choisi. Une analyse de l'impact de la taille des chunks, du reranker ou du query rewriting est attendue.

Contraintes spécifiques

- Évaluation obligatoire sur un jeu de questions construit par l'équipe.
- Mesures minimales : précision, rappel documentaire, qualité de citation, taux d'hallucination, latence.
- Intégration obligatoire avec la mémoire et avec au moins un serveur MCP exposant les ressources.
- Justification détaillée du corpus, du nettoyage et de la segmentation.

Livrables spécifiques

Article d'évaluation RAG, index reproductible, scripts d'ingestion, démonstration avec réponses sourcées.

Questions de recherche possibles

- Quel type de RAG est le plus pertinent pour un corpus universitaire francophone ?
- Quel compromis entre qualité, coût et latence est acceptable ?

Critères d'évaluation prioritaires

Critère	Ce qui est attendu	Poids conseillé
Profondeur scientifique	Problématique claire, hypothèse, analyse critique	35 %
Qualité technique	Architecture, code, reproductibilité, intégration complète	35 %
Évaluation expérimentale	Comparaisons, métriques, interprétation	20 %
Communication	Article, figures, présentation, démonstration	10 %

Projet 5 - Plateforme modulaire de services MCP pour agents IA

Dominante scientifique : MCP.

Problématique

Comment structurer proprement un écosystème d’outils, de ressources et de prompts pour qu’un agent soit modulaire, maintenable et réutilisable ? Ce projet doit dépasser un unique gros serveur fourre-tout et étudier une décomposition propre des services MCP.

Contexte applicatif conseillé

Le système cible comprendra plusieurs serveurs MCP spécialisés : fichiers, mémoire, RAG, Java tools, évaluation, export, monitoring, etc. L’agent consommera ces services de manière dynamique.

Travail de recherche demandé

Comparer deux stratégies : serveur MCP monolithique contre architecture multi-serveurs spécialisés. Étudier découverte des capacités, cohérence des schémas, versionnement, granularité des services et politiques d’accès.

Dimension innovation attendue

La contribution peut porter sur un registre local des services MCP, sur des conventions de description, sur la séparation tools/ressources/prompts, ou sur des mécanismes de fallback entre serveurs.

Contraintes spécifiques

- Au moins 3 serveurs MCP spécialisés.
- Présence obligatoire de tools, ressources et, si possible, prompts.
- Traçabilité des appels et journalisation détaillée.
- Comparaison de maintenabilité, réutilisabilité et qualité de décision.

Livrables spécifiques

Architecture MCP modulaire, documentation technique, article comparatif, démonstrateur de découverte dynamique.

Questions de recherche possibles

- Une architecture multi-serveurs MCP est-elle plus maintenable qu’un unique serveur ?
- Comment normaliser proprement les descriptions et schémas des services ?

Critères d’évaluation prioritaires

Critère	Ce qui est attendu	Poids conseillé
Profondeur scientifique	Problématique claire, hypothèse, analyse critique	35 %
Qualité technique	Architecture, code, reproductibilité, intégration complète	35 %
Évaluation expérimentale	Comparaisons, métriques, interprétation	20 %
Communication	Article, figures, présentation, démonstration	10 %

Projet 6 - Coopération inter-agents via A2A pour résolution de tâches complexes

Dominante scientifique : A2A.

Problématique

Comment faire coopérer plusieurs agents distincts, éventuellement hébergés sur des processus ou services séparés, tout en gérant délégation, coordination, arbitrage et robustesse ? Le projet doit traiter la coopération réelle et non une simple succession de modules internes.

Contexte applicatif conseillé

Le cas d’usage peut être un assistant de projet PFE ou un analyste de dossier complexe où interviennent un agent documentaire, un agent raisonnement, un agent vérification et un agent synthèse.

Travail de recherche demandé

Étudier les stratégies de coopération : demande directe, délégation supervisée, vote, arbitrage par un agent juge, négociation ou protocole hybride. Définir ce que chaque agent publie comme capacités ou carte descriptive.

Dimension innovation attendue

Mettre en place un mécanisme explicable de résolution de conflits entre agents, ou un protocole de confiance basé sur la provenance, l’historique de performance ou le domaine d’expertise déclaré.

Contraintes spécifiques

- Au moins 2 agents réellement distincts et spécialisés.
- Échanges structurés, journalisés et analysables.
- Connexion avec mémoire, RAG et tools exposés via MCP.
- Scénario de panne ou de désaccord obligatoirement traité.

Livrables spécifiques

Article sur coopération inter-agents, démonstration de scénarios A2A, journal de dialogues d’agents, analyse de robustesse.

Questions de recherche possibles

- La coopération de plusieurs agents améliore-t-elle réellement la qualité finale ?
- Comment arbitrer efficacement des réponses contradictoires ?

Critères d’évaluation prioritaires

Critère	Ce qui est attendu	Poids conseillé
Profondeur scientifique	Problématique claire, hypothèse, analyse critique	35 %
Qualité technique	Architecture, code, reproductibilité, intégration complète	35 %
Évaluation expérimentale	Comparaisons, métriques, interprétation	20 %
Communication	Article, figures, présentation, démonstration	10 %

Projet 7 - Système agentique intégré pour l'université : gouvernance, observabilité et performance

Dominante scientifique : intégration globale.

Problématique

Comment construire une plateforme agentique complète, cohérente et gouvernée, capable de gérer mémoire, tools, RAG, MCP et A2A dans un cadre institutionnel ? Le projet doit résoudre un problème d'intégration système et non simplement juxtaposer des composants.

Contexte applicatif conseillé

Le système peut servir de base à un assistant universitaire local : aide aux cours, support projets, recherche dans documents, génération de synthèses, suivi étudiant, ou orientation intelligente.

Travail de recherche demandé

Concevoir un mini "agentic operating system" local avec registre de services, politique mémoire, observabilité, journalisation, métriques, politique de sécurité, gestion des coûts, limitations d'usage et droit à l'oubli.

Dimension innovation attendue

La vraie valeur ajoutée attendue est la gouvernance du système : traçabilité des décisions, provenance des réponses, instrumentation, monitoring, tableaux de bord, mécanismes de contrôle et d'évaluation continue.

Contraintes spécifiques

- La pile complète doit être pleinement intégrée.
- Présence obligatoire d'un tableau de bord de monitoring ou d'un journal exploitable.
- Évaluation sur plusieurs scénarios d'usage réels.
- Analyse explicite des compromis modularité/coût/latence/qualité.

Livrables spécifiques

Prototype intégré, article de synthèse système, démonstration complète, rapport d'observabilité et de performance.

Questions de recherche possibles

- Quels compromis apparaissent quand on cherche à industrialiser un système agentique local ?
- Comment concilier qualité, traçabilité, coût et simplicité de déploiement ?

Critères d'évaluation prioritaires

Critère	Ce qui est attendu	Poids conseillé
Profondeur scientifique	Problématique claire, hypothèse, analyse critique	35 %
Qualité technique	Architecture, code, reproductibilité, intégration complète	35 %
Évaluation expérimentale	Comparaisons, métriques, interprétation	20 %
Communication	Article, figures, présentation, démonstration	10 %

4. Structure conseillée de l'article scientifique

1. Introduction : contexte, motivation, limites des approches existantes et problématique visée.
2. État de l'art synthétique : concepts utiles, approches voisines, limites identifiées.
3. Méthodologie : hypothèse, architecture proposée, composants, jeux de données, outils et protocole.
4. Implémentation : détails de réalisation, organisation du code, outils MCP, mémoire, RAG, agents et protocoles.
5. Expérimentation : scénarios, métriques, variantes comparées, environnement matériel et logiciel.
6. Résultats et discussion : interprétation critique, limites, erreurs observées, apports réels.
7. Conclusion et perspectives : synthèse, améliorations possibles et ouverture scientifique.

5. Structure conseillée du dépôt de code

```
project/  
  README.md  
docs/  
  article.pdf  
  presentation.pptx  
src/  
  orchestrator/  
  agents/  
  memory/  
  rag/  
  mcp_servers/  
  tools/  
tests/  
data/  
scripts/  
requirements.txt ou pom.xml  
demo.sh ou run.bat
```

6. Recommandations finales aux étudiants

- Commencer par clarifier la question de recherche avant de coder.
- Définir des métriques mesurables dès le début du projet.
- Construire rapidement un prototype minimal, puis itérer.
- Documenter toutes les décisions d'architecture et tous les échecs significatifs.
- Prévoir une démonstration réaliste, stable et courte.
- Veiller à la qualité de rédaction scientifique et à la lisibilité du code commenté.